
Running the App Factory

Every step, in order: the weekly rhythm you'll actually live in, and the full sequence for taking a brand-new app from an empty repository to something running on your phone.

How to use this guide

There are two things you do with this system, and they have very different shapes.

Part One is the rhythm: writing a ticket, queueing it, reviewing what came back. You'll do this hundreds of times. It should take five minutes in the evening and ten in the morning. If it starts taking longer, something has gone wrong and Part Three will tell you what.

Part Two is starting a new app. You'll do this maybe three times a year. It's long, it's mostly one-time setup, and it's worth following literally rather than from memory.

The one habit that matters

Everything else in this guide is mechanics. The thing that determines whether this system works for you is **the quality of the tickets you write**. Agent capability is not the limiting factor. A vague ticket produces a confident, wrong answer at 3am, and you find out at 8am. A precise ticket produces something you merge in ninety seconds.

Part One · The weekly rhythm

What's already running

The routine	<code>FPL Advisor overnight pipeline</code> , fires daily at 2:00 AM Singapore time
Model	Sonnet 5, for quota reasons
Batch limit	2 tickets per run
Revision cap	2 attempts before a ticket is blocked for you
Dashboard	<code>claude.ai/code/routines</code>

An empty night is nearly free. If nothing is queued, the run posts "nothing queued" and stops before loading anything. So a daily schedule does not mean daily cost.

What you queue is what you spend.

Evening · queueing work

1 Write the ticket as a GitHub issue

Go to your repo → **Issues** → **New issue** → pick the **Ticket** template. It gives you four sections. Fill in all of them; a placeholder left in place is what bounces a ticket back while you're asleep.

Title format: `<verb> <thing> – <one clause on why>`

The four sections, and what each is really for:

- **Context:** which part of the product brief this serves. One or two sentences.
- **Scope:** what's in, and crucially *what's explicitly out*. The out-of-scope line is what stops a Builder wandering into territory you didn't intend. "Does not add any new stored data" is a good one.
- **Definition of done:** a checklist that Builder and QA can both point at and agree is objectively met. Not "looks good."
- **Notes:** anything ambiguous that you already know the answer to. Better written here than escalated at 3am.

Check the premise before you write

Take thirty seconds to confirm the thing you're asking for isn't already done, and that anything it depends on actually exists. Both of the first test tickets failed this. One asked for work already in the codebase, the other assumed a screen that had never been built. The system caught both, but each burned a cycle.

2 Lint it: the highest-value five minutes in the whole system

In Terminal:

```
cd ~/Projects/fpl-advisor
claude
```

Then paste, replacing **N** with your issue number:

```
Fetch issue #N with `gh issue view N`, then dispatch the
analyst
subagent with the full ticket text. Wait for it; do not
background
it. Report the verdict, the Tier-1 scope check, and the DoD
verifiability note.
```

You get back three things:

- **A verdict:** PASS, or NEEDS EDIT with each problem named.
- **A Tier-1 scope check:** not just whether the ticket *asks* something risky, but whether building it *as described* would force a Tier 1 decision by default.
- **A verifiability note:** which criteria QA won't be able to check without a real device, so you know in advance what lands in the "not tested" list.

Fix what it flags. The whole point is that you're awake right now and won't be at 3am.

3 Queue it

Add the label `status:ready` to the issue. That is all it takes; that label is the only thing that puts work into the pipeline.

Order is ascending issue number. Lowest open number gets picked first. There's no priority field, so file tickets in the order you want them built. Only two get picked per run, so queueing five doesn't get five built.

Morning · the ten-minute review

1 Open the pull request

You'll have a GitHub notification. The PR description is written for exactly this moment.

2 Tap the preview link on the first line, and use the app

This is the actual review. Not the diff, the running app. Poke the thing the ticket was about.

3 Read the paragraph, then two specific parts

Read "what changed." Then read **high-impact decisions**: each should say *because*, and that's where you catch a misread brief in three seconds. Then read **what was not tested**, which tells you what to check yourself.

4 Merge, or write one sentence

To merge: **Ready for review** → **Merge pull request** → **Confirm**. Then **Delete branch**. The issue closes itself.

If something's wrong, comment on the issue saying what, add

`status:ready` back, and it gets picked up next run. Don't fix it by hand.

That hides a problem the system should have caught.

The tell that this has stopped working

If you can't answer "*what did I merge yesterday?*", you've started rubber-stamping. That's the realistic failure of this whole system. Not a dramatic bad merge, but a slow slide into approving without looking. If you notice it, queue fewer tickets rather than reviewing faster.

When a ticket comes back blocked

A blocked issue carries the `status:blocked` label and — more importantly — **a comment with a single question**. You get a notification for the comment. That question is written to be answerable from a phone with no other context.

Three things cause a block:

Cause	What you do
Tier 1 hit	Something needs an account, a credential, real money, or would store personal data. Only you can do it. Do the thing yourself, or change the ticket to avoid needing it, then re-label <code>status:ready</code> .
Revision cap	QA failed the work twice. Read the comment for what kept failing. Usually the ticket was ambiguous, so rewrite it rather than asking again.
Died mid-run	A previous run stopped partway and left a branch with work on it. You'll be asked: resume, restart, or discard. "Discard" is nearly always right: delete the branch and re-queue.

Answer by commenting on the issue. Then either re-label it `status:ready` or close it.

Polish tickets

Normal build tickets deliberately run on the baseline design skill plus the reference file only. The heavy design tooling — Impeccable, the motion skills — is confined to tickets explicitly typed "polish", because those skills load large reference files into context on every pass and that's a tax you'd otherwise pay on every ticket.

So: batch up visual work and run it as its own ticket at a feature milestone. Write "This is a polish ticket" in the Context, and name the surface: "the fixtures screen", not "the app".

Part Two · Starting a new app

Roughly three hours across two sittings. Do it in order. Several steps depend on earlier ones in ways that aren't obvious.

Stage 1 · The repository

1 Create the repo on GitHub

Private, with a README. One repo per app, never a monorepo, because each app has its own hosting project and its own release rhythm.

2 Scaffold the app locally

```
cd ~/Projects
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm install -D vite-plugin-pwa
```

Then configure the PWA plugin in `vite.config.ts` with the app's name, icons and theme colour, so it installs to a home screen properly. Copy this from `fpl-advisor` : it's already correct there.

3 Copy the pipeline files across

These are what make the repo drivable by the factory. From `fpl-advisor` , copy:

- `.claude/agents/` : analyst, builder, qa
- `.claude/settings.json` : the permission allowlist
- `escalation.md` : the canonical tier definitions
- `.github/ISSUE_TEMPLATE/ticket.md`
- `design-reference.md` : then replace its contents for the new app
- `CLAUDE.md` : then rewrite the app-specific parts
- An empty `decisions.md` with its two headings

Commit and push all of it to `main` .

Stage 2 · Hosting and data

4 Connect Vercel

Import the repo at `vercel.com/new`. It auto-detects Vite. Then — and this one is easy to miss — go to **Settings** → **Deployment Protection** and **turn off Vercel Authentication**. If you skip it, every preview link demands a login, which breaks phone review and stops QA from testing the deployed build.

5 Set up Supabase, if the app needs storage

Create the project yourself, since account creation is always owner-only. Put the URL and publishable key into Vercel's environment variables. **Never in the repo**. Use the variable name `VITE_SUPABASE_PUBLISHABLE_KEY`, not the legacy `ANON_KEY` name.

Stage 3 · GitHub wiring

6 Create the four labels

Repo → **Issues** → **Labels** → **New label**, four times. Copy-paste the names exactly. Applying a label that doesn't exist silently *creates* it, so a typo invents a state nothing queries and the ticket vanishes with no error.

<code>status:ready</code>	green	<code>#0E8A16</code>
<code>status:in-progress</code>	yellow	<code>#FBCA04</code>
<code>status:for-review</code>	blue	<code>#1D76DB</code>
<code>status:blocked</code>	red	<code>#D93F0B</code>

7 Install the Claude GitHub App on the new repo

`github.com/apps/claude` → **Configure** → add the new repository. Without it, runs can't reach pull request and deployment data, and every review packet says "Preview: pending".

Stage 4 · The product brief

8 Write `product-brief.md` and commit it to the repo root

This is the document the Analyst consults before reasoning about anything. A silent brief is the single biggest cause of bad overnight decisions. It must state, explicitly:

- **What the app is for**, and who uses it (you).
- **What personal data it may store**: with an explicit *none* if that's the answer. This one prevents a whole class of Tier 1 false alarms.
- **The chosen external data source**, named, with a written reason. This is a Tier 2 decision you make deliberately rather than letting an agent settle it at 3am.
- **Any in-app currency is Tier 3**, if the app has fake money in it. Without this line, the Analyst will block half your backlog.
- **Locale and formatting**: Singapore time, date formats, and so on.

Stage 5 · The routine

9 Create it

At `claude.ai/code/routines` → **New routine**:

Name	Something like <code><App> overnight pipeline</code>
Prompt	Everything below the rule in <code>assets/orchestrator-prompt.md</code>
Model	Sonnet 5
Repositories	The new repo, and only that one
Environment	<code>App Factory</code> : reuse it; add any new domains the app needs
Trigger	Daily, 2:00 AM. Custom cron is available if you want specific nights (note it is evaluated in UTC)
Connectors	Remove every one . They're attached by default with full write access and no prompts

10 Smoke test before trusting it

File one deliberately trivial ticket with a visible result, such as a line of text somewhere. Lint it, label it `status: ready`, hit **Run now**, and watch. You're testing the pipeline, not the app.

Then check: did the run exit cheaply when the queue was empty? Did a draft PR open with all five packet parts? Did the issue close itself on merge? Anything wrong here gets fixed **in the agent definitions**, never by hand-editing the output. A hand-fixed PR teaches you nothing about whether the system works.

Two traps that have already caught us once each

The routine holds its own copy of the prompt. Editing `orchestrator-prompt.md` changes nothing until you re-paste it into the Instructions box. Every edit to that file ends with a re-paste, or it isn't real.

Runs clone the default branch. Changes sitting on a feature branch, or committed but unpushed, are invisible to every run. After changing anything in the repo, confirm with `git log origin/main --oneline -3`.

Part Three · When things go wrong

Symptom	What it means and what to do
Run says "nothing queued" but you queued something	The label is wrong. Check for a typo, because a mistyped label gets created silently. Look at the repo's label list for near-duplicates.
Nothing happened at 2am	Check the routine is Active on the dashboard, and check your daily run allowance at claude.ai/settings/usage .
Run shows green but nothing was built	Green means the session exited without infrastructure error. It says nothing about the task. Open the run and read the transcript.
PR says "Preview: pending"	Vercel hadn't finished when the run polled. The link usually appears in a bot comment on the PR a minute later.
Preview link asks you to log in	Deployment Protection is on. Vercel → project → Settings → Deployment Protection → turn off Vercel Authentication.
Packet is missing one of the five parts	That's a QA-definition bug. Fix .claude/agents/qa.md , push it to <code>main</code> , re-run the ticket. Don't edit the PR text.
An issue is stuck on <code>status:in-progress</code>	A run died mid-ticket. The next run's recovery handles it automatically: branch has commits → blocked with a question; no commits → branch deleted, back to ready.
Same mistake two nights running	Stop re-running. Fix the agent definition or the product brief. The system is telling you a rule is missing.
Tier 1 fired on something harmless, twice	Don't loosen the tiers. That is how the looseness leaks into real cases. Answer the question in the product brief instead.

Part Four · Questions you'll have

How much does a run cost me?

In quota, not money: it all comes from the one Pro subscription, shared with your daytime chat use. A two-ticket night is roughly twelve agent passes. An empty night is nearly free because of the early exit. The practical control is how many tickets you queue, not how often the routine fires.

Can I queue five tickets and have them all built?

No, two per run, deliberately. Two tickets in one run beats one ticket in two runs, because every fresh session pays a fixed cost re-reading the repo and the brief. Queue five and the other three simply wait for following nights.

Why can't I just merge things automatically if QA passed?

Because that gate is the only thing standing between unreviewed code and production, and it costs about ten seconds. QA is good but it has no phone, no taste, and no memory of what you actually wanted.

What if I want to change something mid-run?

You can't, and you shouldn't try. Runs are stateless and unsupervised. Let it finish, then comment on the issue. If it's genuinely urgent, disable the routine on the dashboard.

The Builder wants to use some library I've never heard of. Should I care?

It'll appear in the high-impact decisions section with a *because*, since framework and major library choices are Tier 2. Read the reason. If it doesn't convince you, say so on the issue. Reversing a library choice after ten more tickets is exactly the expense that tier exists to flag.

Can I run two apps at once?

Technically yes, one routine each. Don't, until the first app has proven the cadence. Two apps means two ticket queues, two morning reviews, and double the quota from the same pool.

What happens if a run dies halfway?

Nothing bad. The next run notices an issue stuck on `status:in-progress` and recovers it. Uncommitted work is lost, which is by design: losing a partial draft is cheaper than building on top of work nobody inspected.

Do I need to understand the code it writes?

Not line by line. You need to understand what changed and why, which is what the packet is for. But if you find yourself merging things you couldn't explain in a sentence, that's the review decay signal.

Why is the decision log such a big deal?

Because it's the only record of judgements made while nobody was watching. Six months in, "why is the data shaped like this?" has an answer. During the shakedown the log was accidentally overwritten by a bad write and would have been silently destroyed. The build still passed, QA still passed, the PR still looked clean. It's now protected by an explicit append-and-verify rule.

Something in the platform changed and a step here doesn't match.

Trust the screen, not this document. Scheduled cloud agents are a research preview and several things shifted during setup. When in doubt, run a small diagnostic rather than assuming. That is how the project board question got settled.

Quick reference

The evening, in four lines

1. New issue, Ticket template, fill all four sections
2. `cd ~/Projects/<app> && claude` → lint it with the analyst
3. Fix what it flags
4. Add label: `status:ready`

The morning, in four lines

1. Open the PR, tap the preview link, use the app
2. Read: what changed / decisions + because / what wasn't tested
3. Merge → Delete branch (the issue closes itself)
4. Or: comment one sentence, re-label `status:ready`

Labels

<code>status:ready</code>	Queued. You add this. Nothing else starts work.
<code>status:in-progress</code>	Being built. Set by the run.
<code>status:for-review</code>	PR open, waiting on you.
<code>status:blocked</code>	A question for you: read the comment.
<i>issue closed</i>	Done. Happens automatically when you merge.

Links worth bookmarking

Blocked tickets	<code>github.com/KeshavPeri/fpl-advisor/issues?q=is:open+label:"status:blocked"</code>
Waiting on review	<code>...?q=is:open+label:"status:for-review"</code>
The queue	<code>...?q=is:open+label:"status:ready"</code>
Routine dashboard	<code>claude.ai/code/routines</code>
Production	<code>fpl-advisor-wine.vercel.app</code>

Rules that don't bend

- Merging is always manual.
- No agent creates an account, an API key, or a credential. Ever. That's yours.
- Linting happens in the evening, interactively, never inside the run.
- Fix definitions, not outputs.
- Every high-impact decision states its *because*.

If you only remember one thing: the system's output quality is set by your ticket quality. Everything else here is plumbing that already works.